

event_study

事件研究分析

夏里

Version 0.1.4

2026 年 4 月 20 日

目录

1	简介	7
1.1	功能特点	7
1.2	主要概念	7
1.2.1	事件研究法概述	7
1.2.2	核心指标	8
2	安装	9
2.1	环境要求	9
3	数据准备	11
3.1	数据来源	11
3.2	个股日收益率数据	11
3.3	市场指数数据	12
3.4	事件日数据	12
3.5	停牌数据	13
3.6	合并数据	13
3.7	停牌处理	15
4	事件研究分析函数	17
4.1	mark_event_window 函数	17
4.1.1	函数签名	17
4.1.2	参数说明	17
4.2	event_study 函数	18
4.2.1	函数签名	18
4.2.2	参数说明	18
4.2.3	predict_model 选项详解	19
4.2.4	返回值	19
4.2.5	输出文件	20
4.2.6	完整示例	20

5	可视化函数	21
5.1	EventStudyPlotter 类	21
5.1.1	类初始化	21
5.1.2	plot_from_dataframe 方法	21
5.1.3	plot_comparison 方法	22
5.1.4	plot_car_distribution 方法	22
5.1.5	plot_significance_summary 方法	22
5.1.6	create_dashboard 方法	23
5.2	独立绘图函数	23
5.2.1	plot_from_event_analysis	23
5.2.2	plot_company_event	24
5.2.3	plot_company_comparison	26
5.2.4	list_available_selections	27
6	输出结果详解	29
6.1	df_ar_by_company 列说明	29
6.2	df_aar 列说明	29
6.2.1	基本信息列	30
6.2.2	AAR 相关列	30
6.2.3	中位数 AR 相关列	30
6.2.4	正收益比例相关列	30
6.2.5	CAAR 相关列	30
6.3	显著性标记	31
7	统计检验详解	33
7.1	t 检验	33
7.2	Wilcoxon 符号秩检验	33
7.3	二项符号检验	34
8	常见问题	35
8.1	数据问题	35
8.1.1	事件日超出数据范围	35
8.1.2	交易日不足	35
8.2	可视化问题	35
8.2.1	中文显示问题	35
9	案例分析	37
9.1	研究背景	37
9.1.1	研究假设	37

9.2	数据准备	38
9.3	执行事件研究分析	38
9.4	统计检验结果	39
9.4.1	单事件 CAR 详情	39
9.4.2	AAR 与 CAAR 日度统计	39
9.5	可视化结果	40
9.5.1	单家公司 CAR 走势	40
9.5.2	多家公司对比	42
9.5.3	CAAR 和 AAR 聚合结果	43
9.6	研究结论	43
9.6.1	假设验证	43
9.6.2	综合结论	43
10	参考文献	45

Chapter 1

简介

`event_study` 是一个用于进行金融事件研究分析的 Python 包，提供了基于市场模型、市场调整模型、Fama 三因子模型的事件研究法。

1.1 功能特点

- 支持多种异常收益率计算模型：市场模型、市场调整模型、Fama-French 三因子模型
- 提供丰富的统计检验：t 检验、Wilcoxon 符号秩检验、二项符号检验
- 完整的可视化功能：CAAR/AAR 曲线、置信区间、公司对比图、分布图等
- 支持批量处理多个事件
- 输出详细的 CSV 和 Excel 结果文件
- 支持排除停牌期间的事件

1.2 主要概念

1.2.1 事件研究法概述

事件研究法（Event Study）是一种用于衡量特定事件对资产价格影响的标准方法。通过比较事件窗口期间的实际收益率与预期正常收益率，计算出异常收益率（Abnormal Return, AR），进而评估事件的市场效应。该方法由 Fama、Fisher、Jensen 和 Roll（1969）提出，被广泛应用于金融经济学研究。

事件研究法的基本步骤如下：

1. **定义事件窗口：**确定事件发生的日期以及用于估计正常收益率的估计窗口。

2. **计算异常收益率**：使用正常收益率模型估计预期收益率，然后计算实际收益率与预期收益率之差。
3. **累积异常收益率**：将事件窗口内的异常收益率进行累加。
4. **统计检验**：对累积异常收益率进行 t 检验，判断其是否显著不为零。

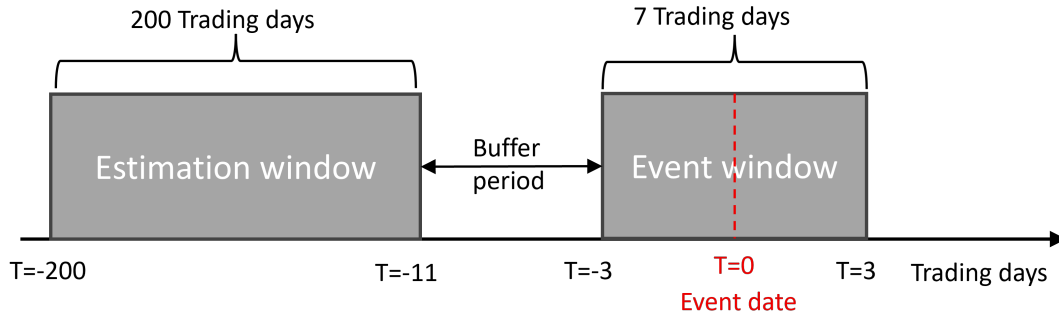


图 1.1: 事件窗口与估计窗口示意图

图1.1中展示了事件研究中常见的时间轴结构，其中， T_0 为事件日（事件发生当天），左侧为估计窗口（用于估计正常收益率模型，图中设置为事件发生前第 200 天至第 11 天，即 200 个交易日），右侧为事件窗口（用于观察事件影响，图中设置为事件发生前第 3 天至发生后第 3 天，即 7 个交易日）。

1.2.2 核心指标

- **AR（异常收益率）**： $AR_{i,t} = R_{i,t} - \hat{R}_{i,t}$
- **CAR（累积异常收益率）**： $CAR_i(\tau_1, \tau_2) = \sum_{t=\tau_1}^{\tau_2} AR_{i,t}$
- **AAR（平均异常收益率）**： $AAR_t = \frac{1}{N} \sum_{i=1}^N AR_{i,t}$
- **CAAR（累积平均异常收益率）**： $CAAR(\tau_1, \tau_2) = \sum_{t=\tau_1}^{\tau_2} AAR_t$
- **ACAR（平均累积异常收益率）**： $ACAR_t = \frac{1}{N} \sum_{i=1}^N CAR_{i,t}$

其中，下标 i 表示公司， t 表示日期， N 表示公司数量。

Chapter 2

安装

2.1 环境要求

- Python 3.8 或更高版本
- pandas
- numpy
- statsmodels
- scipy
- matplotlib（用于可视化）
- seaborn（可选，用于更美观的图表）
- openpyxl（用于 Excel 输出）

Chapter 3

数据准备

事件研究分析需要准备四类数据：个股日收益率数据、市场指数收益率数据、事件日期数据和停牌数据。以下逐一说明各类数据的来源、格式要求及获取方式。

3.1 数据来源

本工具包使用的数据主要从 CSMAR（国泰安数据库）获取，各文件及其对应来源如下：

表 3.1: 数据文件与 CSMAR 数据表对应关系

本工具使用文件	CSMAR 数据表	说明
个股日收益率数据	TRD_Dao3 等	包含每只股票每日的收益率数据
市场指数数据	TRD_Index	包含各类指数的日收益率
事件日数据	用户自行整理	包含股票代码与事件日期
停牌数据	TSR_Stkstat	包含股票停复牌日期信息

3.2 个股日收益率数据

个股日收益率数据应包含以下列：

- Stkcd: 股票代码（如 600028、000001），6 位数字，不足 6 位的前面补零
- Trddt: 交易日期，格式为 YYYY-MM-DD

- **Dretnd**: 考虑分红的日收益率（以小数形式存储，如 0.01 表示 1%）

获取方式：CSMAR 数据库 → 股票市场 → 交易数据 → 个股交易（Day）。

3.3 市场指数数据

市场指数数据应包含以下列：

- **Indexcd**: 指数代码（如 300 代表沪深 300 指数）
- **Trddt**: 交易日期
- **Retindex**: 指数日收益率（以小数形式存储）

获取方式：CSMAR 数据库 → 股票市场 → 指数交易 → 日收益率（综合）。

常用指数代码如下：

表 3.2: 常用市场指数代码

指数代码	指数名称	开始日期
300	沪深 300 (HS_300)	2005-04-08
1	上证综合指数	1991-07-15
399001	深证成份指数	1995-01-23
399106	深证综合指数	1991-04-04

3.4 事件日数据

事件日数据由用户自行整理，格式如下：

- **company_code**: 股票代码（6 位数字）
- **event_date**: 事件日期（YYYY-MM-DD 格式）

示例：

```
1 import pandas as pd
2
3 data = {
4     "company_code": ["601857", "600028", "600938"],
5     "event_date": ["2026-02-28"] * 3
6 }
7 df = pd.DataFrame(data)
8 df.to_csv('event_info.csv', index=False)
```

3.5 停牌数据

停牌数据用于排除事件日处于停牌期间的公司，避免因停牌导致的收益率数据缺失影响分析结果。

数据应包含以下列：

- Stkcd: 股票代码
- Annctime: 公告日期
- Type: 停牌类型
- Suspdte_start: 停牌开始日期
- Suspdte_end: 停牌结束日期
- Resmdate: 复牌日期
- Resmtime: 复牌时间
- Timeperd: 停牌时段
- Reason: 停牌原因

获取方式：CSMAR 数据库 → 股票市场 → 交易日 → 交易状态（TSR_Stkstat）。

3.6 合并数据

数据整合的目标是将个股收益率、市场指数收益率与事件日期合并成一张宽表，格式如下：

- stockid: 股票代码（6 位数字字符串）
- date: 交易日期
- eventdate: 事件日期
- sreturn: 个股日收益率
- mreturn: 市场指数日收益率
- smb: SMB 因子（Fama 三因子模型用，可选）
- hml: HML 因子（Fama 三因子模型用，可选）
- rf: 无风险利率（Fama 三因子模型用，可选）

整合步骤示例如下：

1. 从 SQLite 数据库读取个股收益率数据并整理列名。
2. 从 CSV 文件读取市场指数数据，过滤出所需指数（如沪深 300），提取日期和收益率列。
3. 将个股收益率与市场指数数据按日期内连接（inner join）。
4. 将结果与事件日期表按股票代码连接，保留事件涉及的所有股票的数据。
5. 整理列顺序并保存为 df_eventstudy.csv。

```

1 import pandas as pd
2 import sqlite3
3
4 # 1. 读取个股收益率数据
5 conn = sqlite3.connect(r'公司数据集\日个股回报率数据\stock_daily_data
6     .db')
7 stock_df = pd.read_sql_query(
8     "SELECT Stkcd, Trddt, Dretn FROM daily_data ORDER BY Stkcd,
9     Trddt",
10    conn
11 )
12 conn.close()
13 stock_df.columns = ['stockid', 'date', 'sreturn']
14 stock_df['date'] = pd.to_datetime(stock_df['date'])
15
16 # 2. 读取市场指数数据（沪深300）
17 IDX_FILE = r"公司数据集\指数文件\19901219-20260416\TRD_Index.csv"
18 MARKET_INDEX_CODE = 'HS_300'
19 index_name_mapping = {
20     300: 'HS_300', # 沪深300
21     1: 'SH_ZH', # 上证综合指数
22     399001: 'SZ_CF', # 深证成份指数
23 }
24 idx_raw = pd.read_csv(IDX_FILE, dtype={'Indexcd': int})
25 idx_raw['Index_Mapping'] = idx_raw['Indexcd'].map(index_name_mapping)
26 idx_filtered = idx_raw[idx_raw['Index_Mapping'] == MARKET_INDEX_CODE
    ].copy()
27 idx_filtered['date'] = pd.to_datetime(idx_filtered['Trddt'])
28 idx_filtered = idx_filtered[['date', 'Retindex']].rename(

```

```

27     columns={'Retindex': 'mreturn'})
28 idx_filtered = idx_filtered.set_index('date').sort_index()
29
30 # 3. 读取事件数据
31 EVENT_FILE = r"公司数据集\事件日数据表"
32 events_raw = pd.read_csv(EVENT_FILE + '\event_info.csv')
33 events_raw['eventdate'] = pd.to_datetime(
34     events_raw['event_date'].str.split('_').str[0], format='%Y-%m-%d'
35 )
36 events_raw['stockid'] = events_raw['company_code'].astype(str)
37 events = events_raw[['stockid', 'eventdate']].copy()
38
39 # 4. 合并数据
40 merged = pd.merge(stock_df, idx_filtered.reset_index(),
41                    on='date', how='inner')
42 df_eventstudy = pd.merge(merged, events, on='stockid', how='inner')
43 df_eventstudy = df_eventstudy[['stockid', 'date', 'eventdate',
44                                'sreturn', 'mreturn']]
45 df_eventstudy = df_eventstudy.sort_values(['stockid', 'eventdate', '
46     date'])
47 df_eventstudy.to_csv(r'output\tables\df_eventstudy.csv', index=False)
48 print(df_eventstudy.head())
49 # 输出示例:
50 #   stockid      date  eventdate  sreturn  mreturn
51 # 0  600028  2005-04-08  2026-02-28  0.030588      NaN
52 # 1  600028  2005-04-11  2026-02-28 -0.009132 -0.007996

```

3.7 停牌处理

停牌数据文件格式如下（示例）：

表 3.3: 停牌数据文件格式

列名	类型	说明
Stkcd	str	股票代码（6 位数字）
Annctime	datetime	公告日期
Type	int	停牌类型
Susptime_start	datetime	停牌开始日期
Susptime_end	datetime	停牌结束日期
Resmdate	datetime	复牌日期
Reason	str	停牌原因

在运行 `event_study` 时传入 `suspension_file` 参数即可自动排除事件日处于停牌期间的公司：

```
1 result = es.event_study(  
2     df_eventstudy=df_eventstudy,  
3     event_window_list=[(-5, 10)],  
4     est_window=(-210, -10),  
5     predict_model='market',  
6     suspension_file='TSR_Stkstat.csv'  
7 )
```


Chapter 4

事件研究分析函数

本工具包提供两个核心函数：`mark_event_window` 和 `event_study`。

4.1 `mark_event_window` 函数

标记数据中的估计窗口和事件窗口。

4.1.1 函数签名

```
1 def mark_event_window(group, est_window, event_window)
```

4.1.2 参数说明

表 4.1: `mark_event_window` 参数

参数	类型	说明
<code>group</code>	<code>DataFrame</code>	按 (<code>stockid</code> , <code>eventdate</code>) 分组后的数据
<code>est_window</code>	<code>tuple</code>	估计窗口, 如 (-210, -10)
<code>event_window</code>	<code>tuple</code>	事件窗口, 如 (-5, 10)

该函数会找到事件日 (`date1 = 0`) 对应的索引位置, 然后根据 `est_window` 和 `event_window` 参数标记数据:

- `event_window`: 事件窗口内标记为 1, 否则为 0
- `est_window`: 估计窗口内标记为 1, 否则为 0

4.2 event_study 函数

执行完整的事件研究分析，包括数据预处理、模型估计、异常收益率计算和统计检验。

4.2.1 函数签名

```

1 def event_study(
2     df_eventstudy=None,
3     event_window_list=None,
4     est_window=(-210, -10),
5     predict_model='market',
6     save_path=None,
7     suspension_file=None,
8     min_est_days=200,
9     print_event_details=False,
10    event_detail_days=10,
11    check_stockid=None
12 )

```

4.2.2 参数说明

表 4.2: event_study 主要参数

参数	类型	说明
df_eventstudy	DataFrame	输入数据（必需）
event_window_list	list	事件窗口列表，如 [(-20,-11), (-5,10)]（必需）
est_window	tuple	估计窗口，默认为 (-210, -10)
predict_model	str	预测模型：'market'、'market_adj'、'fama3'
save_path	str	结果保存路径，默认为当前目录
suspension_file	str	停牌数据文件路径（可选）
min_est_days	int	估计窗口最少交易日天数，默认 200
print_event_details	bool	是否打印事件详情，默认 False
event_detail_days	int	打印事件详情时显示的天数，默认 10
check_stockid	str	指定检查的股票代码（可选）

4.2.3 predict_model 选项详解

Market Model（市场模型）

使用市场收益率回归估计预期收益率：

$$R_{i,t} = \alpha_i + \beta_i R_{m,t} + \epsilon_{i,t} \quad (4.1)$$

其中 $R_{i,t}$ 为股票收益率， $R_{m,t}$ 为市场收益率。预期收益率为 $\hat{R}_{i,t} = \hat{\alpha}_i + \hat{\beta}_i R_{m,t}$ 。

使用时设置 `predict_model='market'`，需要数据包含 `mreturn` 列。

Market-Adjusted Model（市场调整模型）

假设预期收益率等于市场收益率：

$$\hat{R}_{i,t} = R_{m,t} \quad (4.2)$$

使用时设置 `predict_model='market_adj'`，需要数据包含 `mreturn` 列。

Fama-French Three Factor Model（Fama 三因子模型）

使用 Fama-French 三因子模型估计预期超额收益率：

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_1 [R_{m,t} - R_{f,t}] + \beta_2 SMB_t + \beta_3 HML_t + \epsilon_{i,t} \quad (4.3)$$

使用时设置 `predict_model='fama3'`，需要数据包含 `smb`、`hml`、`rf` 和 `mreturn` 列。

异常收益率计算方式为：

$$AR_{i,t} = (R_{i,t} - \hat{R}_{i,t}) = (R_{i,t} - R_{f,t}) - (\hat{\alpha}_i + \hat{\beta}_1 [R_{m,t} - R_{f,t}] + \hat{\beta}_2 SMB_t + \hat{\beta}_3 HML_t) \quad (4.4)$$

即使用 OLS 回归残差作为异常收益率。

4.2.4 返回值

`event_study` 返回一个包含四个元素的元组：

1. `t_test_df`: T 检验结果 DataFrame
2. `df_ar_by_company`: 每家公司每天的 AR 和 CAR 数据
3. `df_aar`: 所有公司每天的平均指标（AAR、CAAR 等）
4. `df_exclusion`: 被排除的事件记录

4.2.5 输出文件

函数会在 `save_path` 下生成以下文件：

- `{predict_model}_model_AR_by_company.csv`: 每家公司的 AR 数据
- `{predict_model}_model_AAR_daily.csv`: 每日平均指标
- `{predict_model}_model_ttest.xlsx`: T 检验结果
- `{predict_model}_model_exclusion_records.csv`: 排除记录

4.2.6 完整示例

```
1 import pandas as pd
2 from event_study import event_study
3
4 # 读取数据
5 df_eventstudy = pd.read_csv('output/tables/df_eventstudy.csv')
6
7 # 设置多个事件窗口
8 event_window_list = [
9     (-20, -11),    # 事件前20-11天
10    (-10, -6),      # 事件前10-6天
11    (-5, 10),       # 事件前5天到后10天
12    (11, 20),       # 事件后11-20天
13    (21, 60),       # 事件后21-60天
14 ]
15
16 # 执行分析
17 t_test_df, df_ar_by_company, df_aar, df_exclusion = event_study(
18     df_eventstudy=df_eventstudy,
19     event_window_list=event_window_list,
20     est_window=(-210, -10),
21     predict_model='market',
22     suspension_file='TSR_Stkstat.csv',
23     min_est_days=240,
24     print_event_details=True,
25     check_stockid='600028',
26     save_path='output/tables'
27 )
```

Chapter 5

可视化函数

本工具包提供两类可视化接口：EventStudyPlotter 类（面向对象风格）和独立便捷函数（面向过程风格）。

5.1 EventStudyPlotter 类

该类专门用于事件研究结果的可视化，支持多种绘图功能。

5.1.1 类初始化

```
1 class EventStudyPlotter:
2     def __init__(self, figsize: Tuple[float, float] = (12, 6))
```

初始化绘图器，设置默认图形大小。

5.1.2 plot_from_dataframe 方法

从 df_aar DataFrame 直接绘制 CAAR 曲线图。

```
1 def plot_from_dataframe(
2     self,
3     df_aar: pd.DataFrame,
4     event_window_col: str = 'event_window_str',
5     day_col: str = 'relative_day_adj',
6     car_col: str = 'CAAR',
7     aar_col: str = 'AAR',
8     std_car_col: str = 'Std. E. CAAR',
9     std_aar_col: str = 'Std. E. AAR',
10    title: str = 'Event Study Results',
11    show_aar: bool = False,
```

```

12     show_ci: bool = True,
13     confidence: float = 0.90,
14     figsize: Tuple[float, float] = None,
15     save_path: Optional[str] = None,
16 ) -> plt.Figure

```

5.1.3 plot_comparison 方法

对比多个事件研究结果，将多条 CAAR/AAR 曲线绘制在同一张图上。

```

1 def plot_comparison(
2     self,
3     results_dict: Dict[str, Dict],
4     metric: str = 'CAAR',
5     figsize: Tuple[float, float] = (14, 7),
6     colors: List[str] = None,
7     show_ci: bool = True,
8     confidence: float = 0.90,
9     title: str = 'Event_Study_Comparison',
10 ) -> plt.Figure

```

5.1.4 plot_car_distribution 方法

绘制 CAR 分布直方图和箱线图，用于观察事件窗口内各公司 CAR 的分布情况。

```

1 def plot_car_distribution(
2     self,
3     CAR_values: np.ndarray,
4     event_date: str = '',
5     figsize: Tuple[float, float] = (10, 6),
6     bins: int = 20,
7     kde: bool = True,
8 ) -> plt.Figure

```

5.1.5 plot_significance_summary 方法

绘制显著性热图和显著性比例柱状图，直观展示各事件窗口各日的统计显著性。

```

1 def plot_significance_summary(
2     self,
3     df_aar: pd.DataFrame,

```

```

4     event_window_col: str = 'event_window_str',
5     day_col: str = 'relative_day_adj',
6     car_col: str = 'CAAR',
7     pval_col: str = 'CAAR_P-value',
8     figsize: Tuple[float, float] = (12, 6),
9 ) -> plt.Figure

```

5.1.6 create_dashboard 方法

创建综合仪表盘，包含 CAAR 曲线、AAR 柱状图、ACAR 曲线、显著性热图和 CAR 分布图等多个子图。

```

1 def create_dashboard(
2     self,
3     df_aar: pd.DataFrame,
4     car_values: Optional[np.ndarray] = None,
5     title: str = 'Event_Study_Dashboard',
6     save_path: Optional[str] = None,
7 ) -> plt.Figure

```

5.2 独立绘图函数

除了面向对象的 EventStudyPlotter 类之外，本工具包还提供四个独立的便捷绘图函数。

5.2.1 plot_from_event_analysis

从 event_study 输出的 df_aar DataFrame 直接绘图。

```

1 def plot_from_event_analysis(
2     df_aar: pd.DataFrame,
3     event_window: Optional[str] = None,
4     title: Optional[str] = None,
5     show_aar: bool = False,
6     car_type: str = 'CAAR',
7     show_ci: bool = True,
8     confidence: float = 0.90,
9     figsize: Tuple[float, float] = (9, 6),
10    save_path: Optional[str] = None,
11    ax: Optional[plt.Axes] = None,

```

```
12 ) -> Union[plt.Figure, plt.Axes]
```

- event_window: 事件窗口字符串, 如 '[-5,10]', None 表示自动选择第一个窗口
- car_type: 可选 'CAAR' 或 'ACAR', 默认 'CAAR'
- ax: 可传入外部坐标轴, 用于子图组合
- 置信区间采用双尾 t 分布计算: $\delta = Std.E. \times t.ppf((1 + confidence)/2, df)$

示例:

```
1 from event_study import event_study, plot_from_event_analysis
2
3 t_test_df, df_ar_by_company, df_aar, df_exclusion = event_study(
4     df_eventstudy=df_eventstudy,
5     event_window_list=[(-2, 10)],
6     est_window=(-210, -10),
7     predict_model='market',
8     save_path='output/tables'
9 )
10
11 fig = plot_from_event_analysis(
12     df_aar=df_aar,
13     car_type='CAAR',
14     event_window='[-2, 10]',
15     show_aar=True,
16     show_ci=True,
17     confidence=0.95,
18     title='Impact of War on Chinese Oil Companies',
19     save_path='graphs/CAAR.png'
20 )
```

5.2.2 plot_company_event

绘制特定公司在特定事件日的 AR (柱状图) 和 CAR (折线图), 用于观察单家公司的事件窗口内收益率走势。

```
1 def plot_company_event(
2     df_ar_by_company: pd.DataFrame,
3     stockid: Optional[str] = None,
4     eventdate: Optional[pd.Timestamp] = None,
```



```
5     event_window: Optional[str] = None,
6     title: Optional[str] = None,
7     xlabel: Optional[str] = None,
8     ylabel: Optional[str] = None,
9     show_ci: bool = True,
10    confidence: float = 0.95,
11    figsize: Tuple[float, float] = (9, 6),
12    save_path: Optional[str] = None,
13    ax: Optional[plt.Axes] = None,
14 ) -> None
```

参数说明:

- df_ar_by_company: event_study 输出的 df_ar_by_company
- stockid: 股票代码, None 表示自动选择第一个
- eventdate: 事件日期, None 表示自动选择第一个
- event_window: 事件窗口字符串, 如 '[-5,10]', None 表示自动选择
- xlabel: x 轴标签, None 表示默认'Relative Day', 空字符串" 表示隐藏
- ylabel: y 轴标签, 默认不显示
- ax: 可传入外部坐标轴

示例:

```
1 import pandas as pd
2 from event_study import plot_company_event
3
4 plot_company_event(
5     df_ar_by_company=df_ar_by_company,
6     stockid='600028',
7     eventdate=pd.Timestamp('2026-02-28'),
8     event_window='[-2,10]',
9     show_ci=True,
10    confidence=0.95,
11    save_path='graphs/600028_2026-02-28.png'
12 )
```

5.2.3 plot_company_comparison

将不同公司的 CAR 曲线和 AR 柱状图画在同一张图中，用于对比多家公司在同一事件日的表现。

```

1 def plot_company_comparison(
2     df_ar_by_company: pd.DataFrame,
3     eventdate: pd.Timestamp,
4     event_window_str: str = None,
5     stockids: List[str] = None,
6     figsize: Tuple[float, float] = (14, 7),
7     colors: List[str] = None,
8     title: str = None,
9     bar_width: float = 0.3,
10    ar_alpha: float = 0.5,
11    save_path: str = None,
12 ) -> plt.Figure

```

- eventdate: 事件日期（必需）
- event_window_str: 事件窗口字符串，如 "[-2,10]"
- stockids: 要对比的股票代码列表，None 表示使用该事件日下所有股票
- colors: 颜色列表，支持 seaborn 调色板

示例：

```

1 import seaborn as sns
2 from event_study import plot_company_comparison
3
4 colors = sns.color_palette('Set2', 3)
5 fig = plot_company_comparison(
6     df_ar_by_company=df_ar_by_company,
7     eventdate=pd.Timestamp('2026-02-28'),
8     event_window_str='[-2,10]',
9     stockids=['601857', '600028', '600938'],
10    colors=colors,
11    save_path='graphs/comparison.png'
12 )

```

常用 seaborn 调色板推荐：Set2（柔和，适合 8 种颜色）、husl（鲜明，色相差大）、Paired（成对颜色）、colorblind（色盲友好）。

5.2.4 list_available_selections

列出 `df_ar_by_company` 中所有可用的股票、事件窗口和事件日组合，便于在绘图前确认数据内容。

```
1 def list_available_selections(df_ar_by_company: pd.DataFrame) -> None
```

输出内容包括：股票数量、可用的事件窗口列表、事件日数量，以及示例数据。

Chapter 6

输出结果详解

6.1 df_ar_by_company 列说明

每家公司每天的 AR 和 CAR 数据：

- stockid: 股票代码
- event_window_str: 事件窗口字符串
- date: 交易日期
- eventdate: 事件日期
- relative_day_raw: 相对天数（原始，date - eventdate）
- relative_day_adj: 相对天数（调整后，事件窗口内连续编号）
- AR: 异常收益率
- CAR: 累积异常收益率
- Std. E. AR: AR 标准误
- Std. E. CAR: CAR 标准误
- CAR_T-stat: CAR t 统计量
- CAR_P-value: CAR p 值

6.2 df_aar 列说明

event_study 返回的 df_aar DataFrame 包含以下列：

6.2.1 基本信息列

- `event_window_str`: 事件窗口字符串, 如 `"[-5,10]"`
- `relative_day_adj`: 相对天数 (调整后, 从事件窗口起点开始编号)
- `N_Events`: 事件数量 (公司数量)

6.2.2 AAR 相关列

- `AAR`: 平均异常收益率 (Average Abnormal Return)
- `Std. E. AAR`: AAR 标准误
- `AAR_T-stat`: AAR 的 t 统计量
- `AAR_P-value`: AAR 的 p 值 (双尾检验)

6.2.3 中位数 AR 相关列

- `Median_AR`: 异常收益率中位数
- `Wilcoxon_Z_Median_AR`: Wilcoxon 符号秩检验 Z 统计量
- `Wilcoxon_Z_Median_AR_P-value`: Wilcoxon 检验 p 值

6.2.4 正收益比例相关列

- `Positive_AR(%)`: 正异常收益率的事件占比 (%)
- `Negative_AR(%)`: 负异常收益率的事件占比 (%)
- `Sign_Z_Positive_AR`: 正收益比例的二项符号检验 Z 值
- `Sign_Z_Positive_AR_P-value`: 符号检验 p 值

6.2.5 CAAR 相关列

- `CAAR`: 累积平均异常收益率
- `Std. E. CAAR`: CAAR 标准误
- `CAAR_T-stat`: CAAR 的 t 统计量
- `CAAR_P-value`: CAAR 的 p 值 (双尾检验)

6.3 显著性标记

在输出表格中，使用星号标记显著性水平：

- ***: $p < 0.01$ (99% 置信水平)
- **: $p < 0.05$ (95% 置信水平)
- *: $p < 0.10$ (90% 置信水平)

Chapter 7

统计检验详解

本工具包提供三种统计检验方法。

7.1 t 检验

用于检验 CAAR 是否显著不为零。

t 统计量计算公式：

$$t = \frac{CAAR}{Std.E.CAAR} \quad (7.1)$$

其中 Std.E._CAAR 为 CAAR 的标准误。双尾 p 值计算：

$$p = 2 \times (1 - F_t(|t|, df)) \quad (7.2)$$

自由度使用估计窗口残差自由度之和。

7.2 Wilcoxon 符号秩检验

非参数检验，用于检验中位数 AR 是否显著不为零，不依赖于收益率的正态分布假设。

Wilcoxon W 统计量：

$$W = \sum_{i=1}^N \text{sgn}(AR_i) \cdot \text{rank}(|AR_i|) \quad (7.3)$$

当 N 较大时， W 近似正态分布：

$$Z = \frac{W - \frac{N(N+1)}{4}}{\sqrt{\frac{N(N+1)(2N+1)}{24}}} \quad (7.4)$$

双尾 p 值由标准正态分布计算： $p = 2 \times (1 - \Phi(|Z|))$ 。

7.3 二项符号检验

用于检验正收益比例是否显著不等于 50%（即正负收益是否对称）。

$$Z = \frac{p - 0.5}{\sqrt{0.25/N}} \quad (7.5)$$

其中 p 为正收益比例， N 为样本数量。

Chapter 8

常见问题

8.1 数据问题

8.1.1 事件日超出数据范围

错误信息示例：

事件日 2024-01-15 超出数据集日期范围（数据最新日期：2024-01-10）

解决方法：检查事件日是否写错，或更新价格/指数数据文件到最新日期。

8.1.2 交易日不足

错误信息示例：

事件日前一年交易日不足检查：5 个事件因数据不足被移除

解决方法：事件日距离上市日期不足一年，或期间公司长期停牌。可通过调整 `min_est_days` 参数降低阈值。

8.2 可视化问题

8.2.1 中文显示问题

如果中文字体无法显示，修改 matplotlib 配置：

```
1 import matplotlib.pyplot as plt
2
3 plt.rcParams['font.sans-serif'] = ['SimSun', 'Microsoft YaHei', '
   STSong']
4 plt.rcParams['axes.unicode_minus'] = False
```


Chapter 9

案例分析

本章以美国-以色列-伊朗战争对中国三家石油上市公司股价的影响为案例，展示完整的事件研究分析流程。

9.1 研究背景

伊朗是全球第四大石油探明储量国（日均产能约 400 万桶）和石油输出国组织（OPEC）重要成员国，2026 年 2 月 28 日，美国与以色列联合对伊朗发动军事打击，中东地缘政治局势骤然升级。石油不仅是普通的大宗商品，更是兼具金融属性与战略属性的特殊资源。中东地区集中了全球约 48% 的探明石油储量，该区域的任何军事冲突都会通过以下渠道对国际油价产生冲击：

（1）**供给中断风险**。中东是全球最重要的石油生产与出口枢纽。伊朗扼守霍尔木兹海峡——这条水道承担着全球约 21% 的石油运输量。战争一旦导致伊朗石油产能受损或海峡通行受阻，全球石油供给将在短期内出现显著收缩，恐慌情绪推动价格急剧攀升。

（2）**风险溢价上升**。地缘冲突的爆发会使原油市场的风险溢价显著放大。历史经验表明，从两伊战争（1980—1988 年）、海湾战争（1990—1991 年）到伊拉克战争（2003 年），每一次中东主要军事冲突都伴随国际油价的短期大幅上涨。投资者和石油消费国的不确定性预期是价格波动的重要来源。

9.1.1 研究假设

中国是全球最大的石油进口国，石油对外依存度超过 70%，国际油价波动对我国经济的影响极为显著。三大石油央企——中国石油（601857）、中国石化（600028）和中国海油（600938）——的业务均高度依赖国际原油价格，其股价走势与油价高度相关。基于上述分析，本案例提出以下研究假设：

假设 H1（正向异常收益）：美以伊战争的爆发将导致中国石油企业股价在事件窗口内产生显著正向异常收益率。由于中国高度依赖进口石油，国内炼油与销售企业将受益于油价上涨带来的库存增值与收入增长，市场会对此作出正面反应。

假设 H2（三公司分化）：由于三家公司的业务结构存在差异——中国石油偏重上游勘探开采、中国石化偏重下游炼化销售、中国海油专注于海洋油气开发——三者从油价上涨中获益的程度不同，异常收益率的幅度可能存在明显分化。

本案例以该事件为研究对象，选取以下三家上市公司为研究样本：

- 中国石油（601857）：中国最大的原油和天然气生产商
- 中国石化（600028）：中国最大的炼油和化工产品生产商
- 中国海油（600938）：中国最大的海上油气生产商

三家石油公司的数据均从 CSMAR 数据库获取，包括日个股收益率（TRD_Dao3 系列）和沪深 300 指数收益率（TRD_Index）。

9.2 数据准备

事件日期为 2026-02-28。

数据整合后的格式如下：

	stockid	date	eventdate	sreturn	mreturn
0	600028	2005-04-08	2026-02-28	0.030588	NaN
1	600028	2005-04-11	2026-02-28	-0.009132	-0.007996
2	600028	2005-04-12	2026-02-28	-0.018433	-0.016801
3	600028	2005-04-13	2026-02-28	0.023474	0.022686
...					

9.3 执行事件研究分析

```

1 import pandas as pd
2 from event_study import event_study
3
4 df_eventstudy = pd.read_csv(r'output/tables/df_eventstudy.csv')
5
6 t_test_df, df_ar_by_company, df_aar, df_exclusion = event_study(
7     df_eventstudy=df_eventstudy,
8     event_window_list=[(-2, 10), (-3, 3)],
9     est_window=(-210, -10),
10    predict_model='market',
11    suspension_file=r'公司数据集\交易停复牌数据表\TSR_Stkstat.csv',
12    print_event_details=True,

```

```

13     min_est_days=240,
14     check_stockid='600028',
15     save_path='output/tables'
16 )

```

输出结果显示，三家公司均无停牌记录，估计窗口充足（200 天），分析顺利执行。

9.4 统计检验结果

执行完 `event_study` 函数后，统计检验结果保存在 `output/tables/market_model_AAR_daily.csv` 中。下面以窗口 `[-2, 10]` 为例，对各关键指标进行解读。

9.4.1 单事件 CAR 详情

以中国石化（600028）为例，窗口 `[-2, 10]` 内各交易日的 AR、CAR 及 t 检验结果如下（存于 `output/tables/market_model_AR_by_company.csv`）：

stockid	date	eventdate	sreturn	mreturn	AR	CAR	CAR_t	CAR_p
600028	2026-02-26	2026-02-28	-0.004601	-0.001903	-0.49%	-0.49%	-0.41	0.686
600028	2026-02-27	2026-02-28	-0.004622	-0.003433	-0.47%	-0.96%	-0.56	0.574
600028	2026-03-02	2026-02-28	0.100619	0.003826	9.96%	8.99%	4.29***	0.000
600028	2026-03-03	2026-02-28	0.099859	-0.015390	10.13%	19.12%	7.89***	0.000
600028	2026-03-04	2026-02-28	-0.053708	-0.011442	-5.28%	13.85%	5.11***	0.000
600028	2026-03-05	2026-02-28	-0.050000	0.009792	-5.18%	8.66%	2.92***	0.004

从表中可见，中国石化在事件日前两日（day -2、day -1）的 AR 接近零且不显著，说明 ** 市场未提前泄露消息 **。值得注意的是，2 月 28 日当天并非交易日，因此选择之后最近的 3 月 2 日为事件日 day 0，可以看出当天的 AR 急剧上升至 9.96%，次日（day 1）进一步攀升至 10.13%，CAR 在两天内累计达到 19.12%，t 统计量分别为 4.29 和 7.89，均在 1% 水平上显著。day 2—3 的负向 AR（-5.28% 和 -5.18%）表明短期回调压力，但 CAR 始终保持正值。

三家公司的完整 AR、CAR 序列均已保存至 `output/tables/market_model_AR_by_company.csv`，可进一步按公司进行横向比较分析。

9.4.2 AAR 与 CAAR 日度统计

表 9.1 展示了每日平均异常收益率（AAR）及其 t 检验结果，以及累积平均异常收益率（CAAR）及其检验结果。

表 9.1: 窗口 [-2, 10] AAR 与 CAAR 日度统计检验结果

相对天数	AAR	AAR Std.E.	AAR t 值	AAR p 值	CAAR	CAAR Std.E.	CAAR t 值	CAAR p 值
-2	-1.36%	0.81%	-1.67	0.095	-1.36%	0.81%	-1.67	0.095
-1	0.11%	0.81%	0.14	0.889	-1.24%	1.15%	-1.08	0.279
0	9.83%	0.81%	12.12***	0.000	8.59%	1.41%	6.11***	0.000
1	10.30%	0.81%	12.69***	0.000	18.89%	1.62%	11.64***	0.000
2	-1.80%	0.81%	-2.22**	0.027	17.09%	1.81%	9.42***	0.000
3	-4.38%	0.81%	-5.40***	0.000	12.71%	1.99%	6.39***	0.000
4	-2.80%	0.81%	-3.45***	0.001	9.91%	2.15%	4.61***	0.000
5	4.77%	0.81%	5.87***	0.000	14.68%	2.30%	6.39***	0.000
6	-6.76%	0.81%	-8.33***	0.000	7.92%	2.43%	3.25***	0.001
7	0.50%	0.81%	0.62	0.537	8.42%	2.57%	3.28***	0.001
8	1.00%	0.81%	1.23	0.220	9.42%	2.69%	3.50***	0.000
9	-1.94%	0.81%	-2.39**	0.017	7.48%	2.81%	2.66***	0.008
10	-0.91%	0.81%	-1.12	0.262	6.57%	2.93%	2.24**	0.025

注：***、**、* 分别表示 1%、5%、10% 显著性水平。Std.E. 为标准误。

从表中可以观察到以下关键现象：

- （1）事件日（day 0）的异常收益最为剧烈。三家石油公司的 AAR 在事件日高达 9.83%，远超其他日期，t 统计量为 12.12，在 1% 水平上高度显著。这意味着市场在消息确认当日即作出强烈反应。
- （2）事件次日（day 1）持续高收益。AAR 达到 10.30%，与事件日几乎持平，表明市场对战争消息的消化尚未结束，存在显著的超额收益持续性，支持假设 H2。
- （3）事件后 2—4 天出现回调整理。day 2 的 AAR 为 -1.80%，day 3 为 -4.38%，day 4 为 -2.80%，均为显著负值，反映短期获利了结压力。
- （4）窗口内累积效应显著。从 day 0 到 day 10，CAAR 由 8.59% 逐步攀升至 6.57%，均在 5% 水平上显著为正，整体正向累积效应明确，支持假设 H1。

9.5 可视化结果

9.5.1 单家公司 CAR 走势

分别绘制三家公司的 AR 和 CAR 走势：

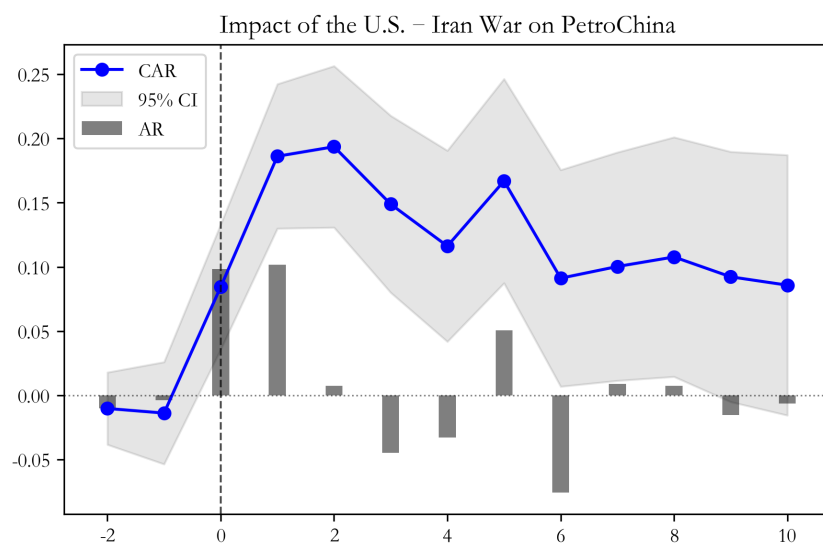


图 9.1: 中国石油（601857）AR 和 CAR 走势

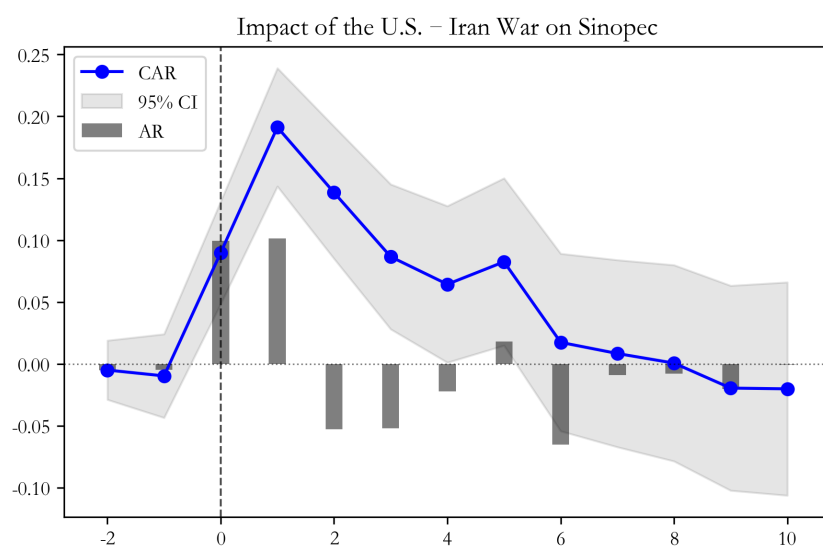


图 9.2: 中国石化（600028）AR 和 CAR 走势

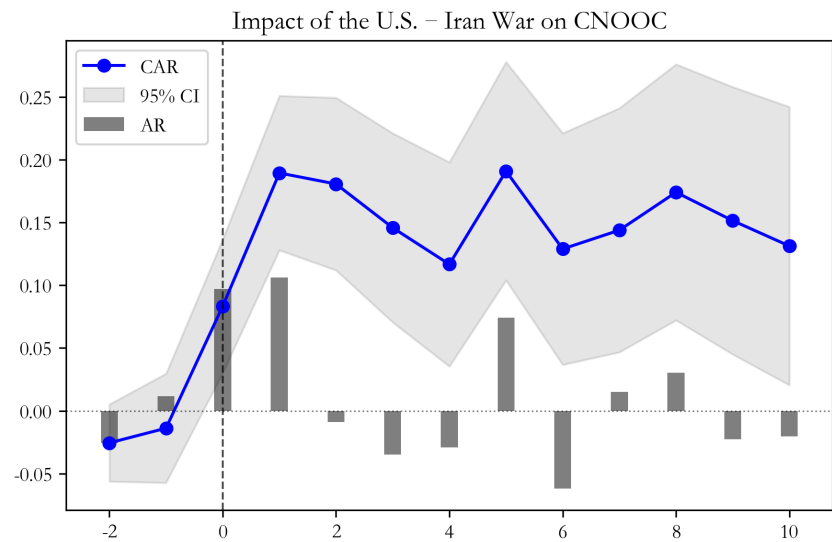


图 9.3: 中国海油（600938）AR 和 CAR 走势

图中蓝色折线为 CAR（累积异常收益率），黑色柱状为 AR（异常收益率），灰色阴影为 95% 置信区间。事件日（相对天数 0）以黑色虚线标记。

9.5.2 多家公司对比

将三家公司的 CAR 曲线和 AR 柱状图画在同一张图中进行对比：

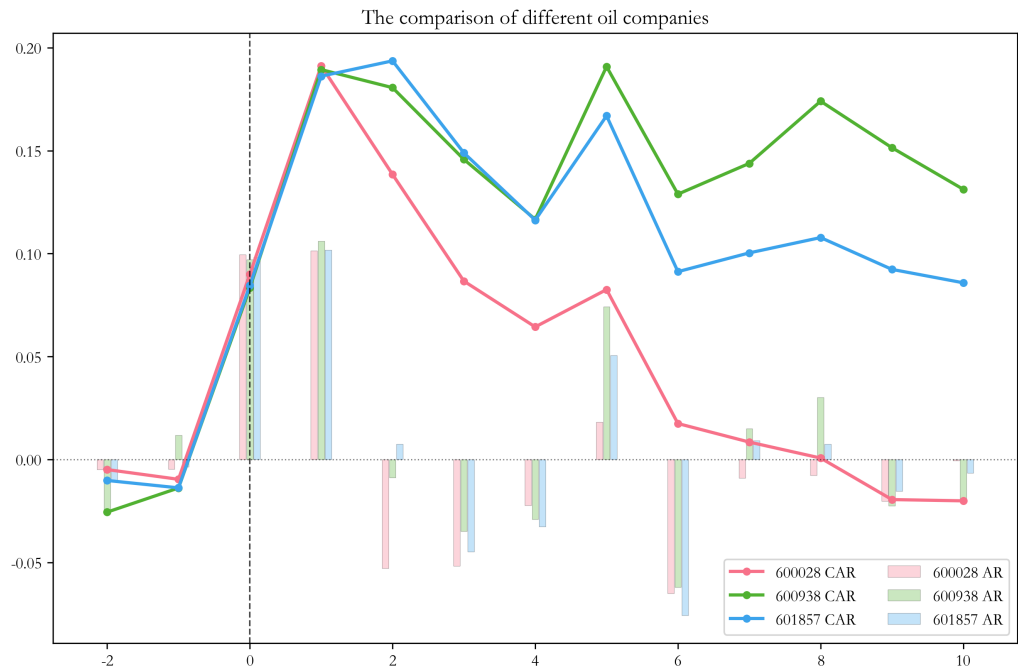


图 9.4: 三家石油公司 AR/CAR 对比

图中清晰展示了三家公司在事件窗口内的差异表现——中国石油在事件后第一天的异常收益率明显高于其他两家公司。

9.5.3 CAAR 和 AAR 聚合结果

绘制三家公司的聚合 CAAR 曲线（95% 置信区间）和 AAR 柱状图：

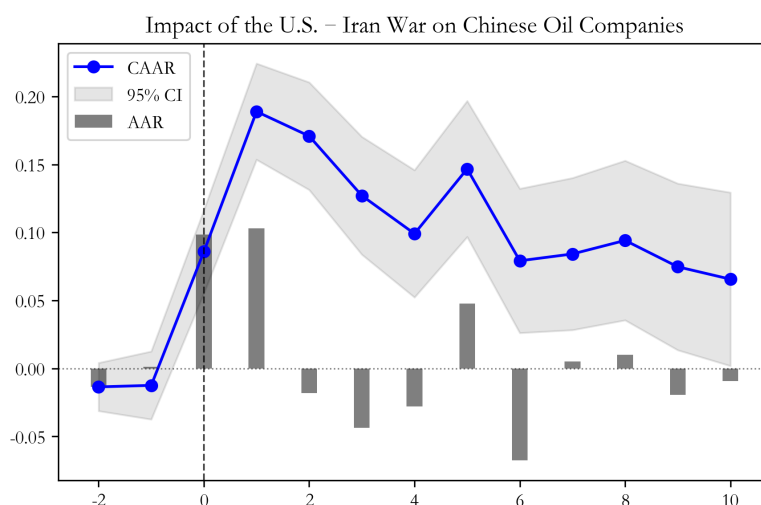


图 9.5: CAAR 曲线（累计平均异常收益率）

9.6 研究结论

基于上述实证结果，本案例对三个研究假设逐一进行验证：

9.6.1 假设验证

假设 H1 验证：正向异常收益。在事件窗口 $[-2, 10]$ 内，事件日（day 0）AAR 高达 9.83%， t 统计量为 12.12，在 1% 显著性水平上拒绝“异常收益为零”的原假设。事件次日（day 1）AAR 进一步攀升至 10.30%，同样高度显著。窗口结束时 CAAR(0,10) 达到 6.57%，在 5% 水平上显著为正。假设 H1 得到充分验证——战争爆发确实引发了三家石油企业显著的正向异常收益。

假设 H2 验证：三公司分化。从三家公司的 CAR 走势（图 9.2—9.1）可以清晰看到：事件日（day 0）中国石油（601857）的 AR 为 9.83%，中国石化（600028）为 9.96%，中国海油（600938）为 9.72%，三者非常接近；但在 day 1，中国石油 AR 达 10.17%，高于中国石化的 10.13% 和中国海油的 10.61%，出现首次分化。更为显著的是，事件后第三天（day 2）中国石油的 AR 仍保持正值（0.74%），而中国石化和中国海油则已转负，这说明三家公司的股价对油价冲击的消化节奏确实存在差异。

9.6.2 综合结论

美以伊战争的爆发通过地缘政治渠道对全球石油市场产生了强烈冲击。三家中国石油上市公司在事件窗口内均获得了显著的正向累积异常收益，其中事件日及次日是收益

最为集中的阶段。三家公司的业务结构差异导致股价反应的时间和幅度有所不同，这一结果表明，地缘政治事件对石油类股票的短期影响具有普遍性，但不同企业因业务特征差异而呈现异质性反应。

Chapter 10

参考文献

- Fama, E. F., Fisher, L., Jensen, M. C., and Roll, R. (1969). “The Adjustment of Stock Prices to New Information.” *International Economic Review*, 10(1), 1-21.
- MacKinlay, A. C. (1997). “Event Studies in Economics and Finance.” *Journal of Economic Literature*, 35(1), 13-39.
- Fama, E. F. and French, K. R. (1992). “The Cross-Section of Expected Stock Returns.” *The Journal of Finance*, 47(2), 427-465.
- Brown, S. J. and Warner, J. B. (1985). “Using Daily Stock Returns: The Case of Event Studies.” *Journal of Financial Economics*, 14(1), 3-31.

本代码重点参考了以下两个 Python 事件研究包的设计思路：

- **myeventstudy** (<https://pypi.org/project/myeventstudy/>): 提供了简洁的事件研究框架，本代码在数据处理流程和模型估计方面借鉴了其设计思想。
- **eventstudy** (<https://pypi.org/project/eventstudy/>, Jean-Baptiste Lemaire): 提供了面向对象的事件研究接口 (Single 类和 Multiple 类)，本代码在事件窗口标记和异常收益率计算逻辑上参考了其实现方式。